Node.js Microservices Code Walkthrough

A teacher-style explanation of the actual code files we added in the JR Jardinage microservices branch.

Branch: `nodejs-microservices`

Main folder: `node-services/`

---

1. How to Read the Node Code

Each service follows almost the same shape. Once you understand one file, the others become much easier.

```
1. Import dependencies
2. Create the Express app
3. Read environment variables
4. Register middleware
5. Define helper functions
6. Define routes
7. Define error handler
8. Start the server
```

> **Teacher note:** A route is a backend endpoint. For example, `POST /api/login` is a route. A handler is the function that runs when that route is called.

2. API Gateway Code

File: `node-services/api-gateway/src/server.js`
Job: receive frontend requests and forward them to the right service.

Important imports

```
const cors = require("cors");
const express = require("express");
const morgan = require("morgan");
const { createProxyMiddleware } = require("http-proxy-middleware");
```

- `express`: creates the API server.

- `cors`: allows the React frontend to call the backend from another port.
- `morgan`: logs requests in the terminal.
- `http-proxy-middleware`: forwards requests to another service.

Service URLs

```
const authServiceUrl = process.env.AUTH_SERVICE_URL || "http://auth-service:3001";
const appointmentServiceUrl =
  process.env.APPOINTMENT_SERVICE_URL || "http://appointment-service:3002";
```

The gateway does not contain login logic or appointment logic. It only knows where those services are.

Proxy route groups

```
app.use([
  "/api/signup",
  "/api/login",
  "/api/admin/login"
], createProxyMiddleware(proxyOptions(authServiceUrl)));
```

This means: if the request path starts with one of those routes, send it to the auth service.

> **Key idea:** the gateway keeps the public API stable. React still calls `/api/login`, but internally the request goes to `auth-service`.

3. Shared Database Code

File: `node-services/shared/db.js`
Job: connect to MySQL and provide a reusable `query` helper.

Connection pool

```
const pool = mysql.createPool({
  host: process.env.DB_HOST || "127.0.0.1",
  port: Number(process.env.DB_PORT || 3306),
  user: process.env.DB_USER || "jardinage",
  password: process.env.DB_PASSWORD || "password123",
  database: process.env.DB_NAME || "jardinage_db"
});
```

A pool means Node keeps a group of reusable database connections. This is better than opening and closing a connection for every request.

The query helper

```
async function query(sql, params = []) {
  const [rows] = await pool.execute(sql, params);
  return rows;
}
```

The important part is that values are passed separately through `params`. That protects against SQL injection.

Database initialization

`initDatabase()` creates the tables if they do not already exist. It creates:

- `admins`
- `customers`
- `service_requests`
- `availability`

4. Shared Auth Code

File: `node-services/shared/auth.js`
Job: create JWTs, verify JWTs, and protect routes.

Generate token

```
function generateToken(payload) {
  return jwt.sign(payload, secret, { expiresIn: "24h" });
}
```

The payload contains data like:

```
{ customer_id: 12, role: "customer" }
```

Admin middleware

```
function requireAdmin(req, res, next) {
  const payload = verifyToken(getBearerToken(req));

  if (!payload) return res.status(401).json({ error: "Missing token" });
  if (payload.role !== "admin") return res.status(403).json({ error: "Unauthorized" });

  req.auth = payload;
  return next();
}
```

`next()` means: the user passed the check, so continue to the real route.

5. Auth Service Code

File: `node-services/auth-service/src/server.js`
Job: own customers, login, password reset, and admin customer management.

Password hashing

```
const passwordHash = await bcrypt.hash(password, 12);
```

We never store raw passwords. We store a hash. During login, we compare the typed password against that hash:

```
const passwordMatches = await bcrypt.compare(password, customer.password);
```

Signup route flow

1. Read `name`, `email`, `password`, and `phone`.
2. Reject missing fields.
3. Reject invalid email format.
4. Check if the email already exists.
5. Hash the password.
6. Create a customer or upgrade a public-request customer.

Login route flow

1. Find the customer by email.
2. Reject if no customer exists.
3. Compare the password with `bcrypt.compare`.
4. If `must_change_password` is true, tell frontend to force password change.
5. Otherwise return a JWT token.

```
return res.json({
  message: "Login successful",
  token: generateToken({ customer_id: customer.id, role: "customer" }),
  role: "customer",
  customer_id: customer.id
});
```

Admin customer creation

The admin can create two types of customers:

Type	Meaning
Internal customer	Stored in admin dashboard, but cannot log in yet.
Login account	Gets a temporary password by email and must change it on first login.

6. Appointment Service Code

File: `node-services/appointment-service/src/server.js`
Job: own requests, appointments, scheduling, cancellations, and blocked dates.

Formatting helper

```
function serviceRequestRow(row) {
  return {
    id: row.id,
    customer_id: row.customer_id,
    customer_name: row.customer_name,
    status: row.status,
    scheduled_start: iso(row.scheduled_start)
  };
}
```

This turns raw database rows into frontend-friendly JSON.

Customer service request route

```
app.post(["/api/service-requests", "/api/appointments"], async (req, res) => {
  // validate customer and date
  // insert pending service request
});
```

This supports both routes because the original Flask app had both during development.

Public quote route

This route is special because the visitor may not have an account.

1. Validate name, phone, address, and preferred date.
2. If email belongs to an existing account, ask user to log in.
3. Otherwise create or reuse a non-active customer.
4. Create a pending service request.
5. Ask notification service to email the admin.

Admin scheduling

The shared `saveAppointment` function supports two cases:

Case	What happens
`request_id` exists	Admin confirms an existing customer request.
`customer_id` exists	Admin creates a new appointment directly from the calendar.

```
if (request_id) {
  // schedule existing request
}

if (customer_id) {
  // create direct appointment
}
```

---

7. Notification Service Code

File: `node-services/notification-service/src/server.js`
Job: send email for the other services.

Why it returns null locally

```
if (!process.env.MAIL_USERNAME || !process.env.MAIL_PASSWORD) {
  return null;
}
```

This allows local Docker testing even if you did not configure Gmail/SMTP yet.

Email route

```
app.post("/internal/email", async (req, res) => {
  const { to, subject, text } = req.body || {};
  // validate fields
  // send email or skip in local mode
});
```

This route is called by backend services, not directly by React.

8. Dockerfiles

Each service has its own Dockerfile. They follow this pattern:

```
FROM node:20-alpine
WORKDIR /app/service-name
COPY package*.json ./
RUN npm install --omit=dev
COPY src ./src
CMD ["npm", "start"]
```

Meaning:

- Start from a small Node.js image.
- Copy dependencies first so Docker can cache install steps.
- Copy source code.
- Run the service with `npm start`.

9. Docker Compose Code

`docker-compose.yml` is the orchestra. It starts all containers and gives each service its environment variables.

```
api-gateway:
  environment:
    AUTH_SERVICE_URL: http://auth-service:3001
    APPOINTMENT_SERVICE_URL: http://appointment-service:3002
```

Inside Docker, services can call each other by service name, like `auth-service` or `appointment-service`.

Depends on

```
depends_on:
  db:
    condition: service_healthy
```

This tells Docker: do not start the service until the database is ready.

10. Code Reading Checklist

When you open any Node service file, ask these questions:

1. What domain does this service own?
2. Which environment variables does it need?
3. Which routes are public?
4. Which routes are protected by `requireAdmin` or `requireCustomer`?
5. Which database tables does it read or write?
6. Does it call another service?
7. What JSON shape does it return to the frontend?

> **Best mental model:** route handler = receive request, validate, use helpers/database, return JSON.

11. Study Exercise

Pick one route and explain it in five sentences. Example:

```
POST /api/public/service-requests
```

1. This route belongs to the appointment service.
2. It receives a quote request from a visitor without an account.
3. It creates or reuses a non-active customer record.
4. It creates a pending service request.
5. It asks notification service to email the admin.

Generated for JR Jardinage, branch `nodejs-microservices`.